

Unidad Temática 4. Bibliotecas de Procesamiento Numérico

Objetivo: Construye sistemas mediante el uso de bibliotecas de procesamiento numérico para crear soluciones con capacidad de análisis de datos.

4.1 Retos del Análisis de Datos

El análisis de datos es el proceso de inspeccionar, limpiar, transformar y modelar datos con el objetivo de descubrir información útil, llegar a conclusiones y apoyar la toma de decisiones. Presenta múltiples retos en la práctica:

Principales retos:

- Volumen y velocidad: gestionar grandes volúmenes de datos de forma eficiente.
- Calidad de datos: valores faltantes, duplicados o inconsistentes.
- Heterogeneidad: distintos formatos (CSV, JSON, bases de datos, APIs).
- Escalabilidad: soluciones que funcionen con datos pequeños y grandes.
- Interpretación: transformar resultados numéricos en decisiones comprensibles.

Ciclo del análisis de datos:

1. Recolección de datos
2. Limpieza y preprocesamiento
3. Exploración y análisis (EDA)
4. Modelado o transformación
5. Visualización e interpretación
6. Comunicación de resultados

4.2 Biblioteca de Análisis de Datos: NumPy y Pandas

Python cuenta con potentes bibliotecas para el análisis numérico. Las dos más utilizadas son NumPy y Pandas.

4.2.1 NumPy — Cómputo Numérico

NumPy (Numerical Python) es la biblioteca fundamental para el cómputo científico en Python. Proporciona objetos de arreglo multidimensional (ndarray) y funciones matemáticas de alto rendimiento.

Conceptos básicos de NumPy:

- ndarray: estructura de datos principal (arreglo N-dimensional).
- dtype: tipo de dato de los elementos del arreglo.

- shape: dimensiones del arreglo (filas, columnas).
- Broadcasting: operaciones entre arreglos de distintas formas.
- Vectorización: operaciones sobre todos los elementos sin ciclos explícitos.

Instalación:

```
pip install numpy
```

Ejemplo 1 — Arreglos básicos y operaciones elementales:

```
import numpy as np

# Crear arreglos
a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])

# Operaciones vectorizadas (sin ciclo for)
suma = a + b          # [11, 22, 33, 44, 55]
producto = a * b      # [10, 40, 90, 160, 250]
raiz = np.sqrt(a)     # [1.0, 1.41, 1.73, 2.0, 2.23]

print('Suma:', suma)
print('Producto:', producto)
print('Raíz cuadrada:', raiz)
```

Ejemplo 2 — Arreglos 2D (matrices) y estadísticas:

```
import numpy as np

# Matriz 3x3 de calificaciones (3 alumnos, 3 materias)
calificaciones = np.array([
    [85, 90, 78],
    [92, 88, 95],
    [70, 75, 80]
])

print('Forma:', calificaciones.shape)          # (3, 3)
print('Promedio general:', calificaciones.mean()) # 83.66
print('Promedio por alumno:', calificaciones.mean(axis=1))
print('Máximo por materia:', calificaciones.max(axis=0))
print('Desviación estándar:', calificaciones.std())
```

Ejemplo 3 — Generación de datos y álgebra lineal:

```
import numpy as np

# Generar 100 valores uniformemente espaciados entre 0 y 2*pi
x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x)

# Arreglo aleatorio
datos = np.random.randint(1, 101, size=(5, 5))

# Producto matricial
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
producto_matricial = np.dot(A, B)
print('A x B =\n', producto_matricial)
```

4.2.2 Pandas — Análisis y Manipulación de Datos

Pandas es la biblioteca estándar para el análisis y manipulación de datos tabulares en Python. Provee dos estructuras de datos principales: Series (1D) y DataFrame (2D).

Conceptos básicos de Pandas:

- Series: arreglo etiquetado unidimensional.
- DataFrame: tabla bidimensional con índice y columnas nombradas.
- Index: etiquetas de filas que permiten acceso eficiente.
- NaN: valor Not a Number, usado para datos faltantes.
- Método apply(): aplica funciones elemento a elemento o por columna/fila.

Instalación:

```
pip install pandas
```

Ejemplo 4 — Crear y explorar un DataFrame:

```
import pandas as pd

# Crear DataFrame con datos de estudiantes
datos = {
    'nombre': ['Ana', 'Luis', 'María', 'Carlos', 'Elena'],
    'edad': [20, 22, 21, 23, 20],
    'promedio': [9.2, 8.5, 9.7, 7.8, 8.9],
    'aprobado': [True, True, True, False, True]
}

df = pd.DataFrame(datos)
print(df.head())      # Primeras 5 filas
print(df.info())      # Tipos de datos y valores no nulos
print(df.describe())  # Estadísticas descriptivas
```

Ejemplo 5 — Filtrado, selección y transformación:

```
import pandas as pd

df = pd.DataFrame({
    'producto': ['Laptop', 'Mouse', 'Teclado', 'Monitor', 'Audífonos'],
    'precio': [15000, 350, 800, 6000, 1200],
    'stock': [10, 50, 30, 8, 25]
})

# Filtrar productos con precio mayor a 1000
caros = df[df['precio'] > 1000]
print(caros)

# Nueva columna: valor total en inventario
df['valor_total'] = df['precio'] * df['stock']

# Ordenar por precio descendente
df_ordenado = df.sort_values('precio', ascending=False)
print(df_ordenado)
```

Ejemplo 6 — Manejo de valores faltantes:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'sensor': ['A', 'B', 'C', 'D', 'E'],
    'temperatura': [22.5, np.nan, 24.1, np.nan, 21.8],
    'humedad': [55, 60, np.nan, 58, 62]
})

print('Valores faltantes por columna:')
print(df.isnull().sum())

# Rellenar NaN con la media de la columna
df['temperatura'] = df['temperatura'].fillna(df['temperatura'].mean())
df['humedad'] = df['humedad'].fillna(df['humedad'].median())
print(df)
```

Ejemplo 7 — Agrupamiento (GroupBy) y agregación:

```
import pandas as pd

ventas = pd.DataFrame({
    'region': ['Norte', 'Sur', 'Norte', 'Sur', 'Centro', 'Centro'],
    'vendedor': ['Ana', 'Luis', 'María', 'Pedro', 'Sofía', 'Juan'],
    'ventas': [120000, 85000, 95000, 110000, 75000, 130000]
})

# Agrupar por región y calcular estadísticas
resumen = ventas.groupby('region')['ventas'].agg(['sum', 'mean', 'max'])
resumen.columns = ['Total', 'Promedio', 'Máximo']
print(resumen)
```

Unidad Temática 5. Bibliotecas de Visualización

Objetivo: Construye sistemas mediante el uso de bibliotecas de visualización para crear soluciones con capacidad de graficar datos y/o resultados.

5.1 Retos de la Visualización de Datos

La visualización de datos convierte números y tablas en gráficas e infografías comprensibles. Sin embargo, presenta sus propios desafíos:

Principales retos:

- Selección del tipo de gráfica adecuada para cada tipo de dato.
- Claridad: evitar gráficas sobrecargadas o engañosas.
- Escalabilidad: visualizar miles o millones de puntos eficientemente.
- Interactividad: permitir al usuario explorar los datos dinámicamente.
- Accesibilidad: colores y contrastes aptos para distintas audiencias.

Tipos de gráficas y cuándo usarlas:

Tipo de Gráfica	Uso recomendado	Ejemplo de datos
Barras / Columnas	Comparar categorías	Ventas por región
Líneas	Tendencias en el tiempo	Temperatura por mes
Dispersión (scatter)	Correlación entre variables	Peso vs estatura
Histograma	Distribución de frecuencias	Edades de usuarios
Pastel (pie)	Proporciones del todo	Participación de mercado
Caja (boxplot)	Distribución y outliers	Calificaciones por grupo

5.2 Biblioteca de Visualización de Datos

Python ofrece múltiples bibliotecas de visualización. Matplotlib es la base; Seaborn agrega estética estadística; Plotly habilita gráficas interactivas.

5.2.1 Matplotlib — Visualización Fundamental

Matplotlib es la biblioteca de visualización más extensa de Python. Permite crear desde gráficas simples hasta figuras científicas complejas con control total sobre cada elemento.

Conceptos básicos:

- Figure: contenedor principal de la gráfica.
- Axes: área donde se dibuja (puede haber varios en una Figure).
- `plt.show()`: muestra la gráfica en pantalla.
- `plt.savefig()`: guarda la gráfica como imagen.

- Subplots: múltiples gráficas en una sola figura.

Instalación:

```
pip install matplotlib
```

Ejemplo 8 — Gráfica de líneas (temperatura mensual):

```
import matplotlib.pyplot as plt

meses = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun',
         'Jul', 'Ago', 'Sep', 'Oct', 'Nov', 'Dic']
temperatura = [12, 14, 17, 20, 24, 27, 29, 28, 25, 21, 16, 13]

plt.figure(figsize=(10, 5))
plt.plot(meses, temperatura, marker='o', color='tomato',
         linewidth=2, label='Temperatura °C')
plt.title('Temperatura Promedio Mensual - Toluca')
plt.xlabel('Mes')
plt.ylabel('Temperatura (°C)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('temperatura.png', dpi=150)
plt.show()
```

Ejemplo 9 — Gráfica de barras (ventas por departamento):

```
import matplotlib.pyplot as plt
import numpy as np

departamentos = ['Sistemas', 'Ventas', 'RRHH', 'Finanzas', 'Operaciones']
ventas_q1 = [45000, 72000, 18000, 31000, 58000]
ventas_q2 = [52000, 68000, 21000, 35000, 61000]

x = np.arange(len(departamentos))
ancho = 0.35

fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - ancho/2, ventas_q1, ancho, label='Q1', color='steelblue')
bars2 = ax.bar(x + ancho/2, ventas_q2, ancho, label='Q2', color='coral')

ax.set_title('Ventas por Departamento (Q1 vs Q2)')
ax.set_xlabel('Departamento')
ax.set_ylabel('Ventas ($)')
ax.set_xticks(x)
ax.set_xticklabels(departamentos)
ax.legend()
ax.bar_label(bars1, padding=3)
ax.bar_label(bars2, padding=3)
plt.tight_layout()
plt.show()
```

Ejemplo 10 — Subplots múltiples:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 100)
```

```

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle('Funciones Trigonométricas', fontsize=14, fontweight='bold')

axes[0, 0].plot(x, np.sin(x), 'b-')
axes[0, 0].set_title('Seno')

axes[0, 1].plot(x, np.cos(x), 'r-')
axes[0, 1].set_title('Coseno')

axes[1, 0].plot(x, np.tan(x), 'g-')
axes[1, 0].set_ylim(-5, 5)
axes[1, 0].set_title('Tangente')

axes[1, 1].plot(x, np.sin(x) + np.cos(x), 'm-')
axes[1, 1].set_title('Sen + Cos')

for ax in axes.flat:
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

5.2.2 Seaborn — Visualización Estadística

Seaborn está construida sobre Matplotlib y proporciona una interfaz de alto nivel para crear gráficas estadísticas atractivas con menos código. Se integra perfectamente con DataFrames de Pandas.

Conceptos básicos:

- **hue:** parámetro para colorear por categoría.
- **palette:** conjunto de colores predefinidos.
- **FacetGrid:** cuadrícula de gráficas por subgrupos.
- **Estimadores:** barras de error, intervalos de confianza automáticos.

Instalación:

```
pip install seaborn
```

Ejemplo 11 — Histograma y distribución:

```

import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Simular calificaciones de un grupo
np.random.seed(42)
calificaciones = np.random.normal(loc=7.5, scale=1.2, size=100)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Histograma con curva KDE
sns.histplot(calificaciones, bins=15, kde=True,
             color='steelblue', ax=axes[0])
axes[0].set_title('Distribución de Calificaciones')
axes[0].set_xlabel('Calificación')

```

```
# Boxplot
sns.boxplot(y=calificaciones, color='lightcoral', ax=axes[1])
axes[1].set_title('Boxplot de Calificaciones')

plt.tight_layout()
plt.show()
```

Ejemplo 12 — Mapa de calor (heatmap):

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Correlación entre variables de sensores IoT
np.random.seed(0)
datos = pd.DataFrame({
    'Temperatura': np.random.normal(25, 3, 100),
    'Humedad': np.random.normal(60, 8, 100),
    'Presión': np.random.normal(1013, 5, 100),
    'CO2': np.random.normal(400, 50, 100),
    'Luz': np.random.normal(500, 100, 100),
})

correlacion = datos.corr()

plt.figure(figsize=(8, 6))
sns.heatmap(correlacion, annot=True, fmt='.2f',
            cmap='coolwarm', center=0, linewidths=0.5)
plt.title('Matriz de Correlación - Sensores IoT')
plt.tight_layout()
plt.show()
```

Ejemplo 13 — Gráfica de dispersión con categorías:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

np.random.seed(10)
n = 80
df = pd.DataFrame({
    'horas_estudio': np.random.uniform(1, 10, n),
    'calificacion': np.random.uniform(5, 10, n),
    'turno': np.random.choice(['Matutino', 'Vespertino'], n),
    'aprobado': np.random.choice([True, False], n)
})

sns.scatterplot(data=df, x='horas_estudio', y='calificacion',
                hue='turno', style='aprobado',
                palette='Set1', s=80)
plt.title('Horas de Estudio vs Calificación por Turno')
plt.xlabel('Horas de Estudio Semanal')
plt.ylabel('Calificación Final')
plt.legend(title='Turno')
plt.grid(True, alpha=0.3)
plt.show()
```


5.2.3 Plotly — Visualización Interactiva

Plotly permite crear gráficas interactivas (zoom, hover, selección) que se renderizan en el navegador o en Jupyter Notebooks. Es ideal para dashboards y aplicaciones web.

Instalación:

```
pip install plotly
```

Ejemplo 14 — Gráfica de líneas interactiva:

```
import plotly.graph_objects as go
import numpy as np

meses = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun',
         'Jul', 'Ago', 'Sep', 'Oct', 'Nov', 'Dic']
ventas_2023 = [42, 55, 60, 48, 72, 85, 90, 78, 65, 70, 80, 95]
ventas_2024 = [50, 62, 71, 55, 80, 88, 95, 83, 70, 75, 88, 102]

fig = go.Figure()
fig.add_trace(go.Scatter(x=meses, y=ventas_2023,
                        mode='lines+markers', name='2023',
                        line=dict(color='royalblue', width=2)))
fig.add_trace(go.Scatter(x=meses, y=ventas_2024,
                        mode='lines+markers', name='2024',
                        line=dict(color='tomato', width=2)))

fig.update_layout(
    title='Ventas Mensuales — Comparativo Anual',
    xaxis_title='Mes',
    yaxis_title='Ventas (miles $)',
    hovermode='x unified'
)
fig.show()
```

Resumen de Bibliotecas por Unidad

Unidad 4 — Procesamiento numérico:

- NumPy: arreglos, vectorización, álgebra lineal, funciones matemáticas.
- Pandas: DataFrames, limpieza, filtrado, agrupamiento, análisis descriptivo.

Unidad 5 — Visualización:

- Matplotlib: gráficas base, subplots, control total del estilo.
- Seaborn: histogramas, boxplots, heatmaps, scatter con categorías.
- Plotly: gráficas interactivas, hover, zoom, ideal para dashboards.